Entregar y mantener sin sustos

Cuánto cuesta de verdad, claves que no se filtran, publicar sin abrir grifos ni pisar lo publicado, y auditar lo que ya está andando.

Jonathan Rodríguez
Construir herramientas con IA, sin ser programador

Antes de empezar

En las tres guías anteriores construimos una herramienta para una tienda de café (ficticia): clasifica las reseñas del mes, escribe un informe que no inventa cifras y corre sola el día 1 con topes y permisos. Hasta aquí, la herramienta es tuya y la usas tú.

Esta última guía es sobre lo que pasa cuando **otros la usan** y cuando **pasan los meses**. Lo curioso es que casi nada de lo que falla en esta etapa tiene que ver con la IA. Falla la plata, fallan las claves, falla la publicación y falla el tiempo.

Riesgo	Cómo se ve	Parte
La plata	La factura del mes triplica lo esperado y nadie sabe por qué	1
Las claves	Alguien está usando tu cuenta de IA, y pagas tú	2
La publicación	Un formulario abierto en internet se vuelve un grifo a tu presupuesto	3
La entrega	Al publicar la versión nueva pisas la que ya estaba funcionando	4
El tiempo	Los precios, los modelos y las cifras que citas cambian, y la herramienta sigue diciendo lo de hace seis meses	5

Qué te llevas

- Cómo calcular el costo real de una herramienta, no solo el de la IA, y qué lo mueve.
- Un registro de costos por corrida, en Python.
- Cómo guardar y leer claves sin que se filtren, y qué hacer si se filtran.
- La arquitectura mínima para publicar algo con IA sin regalar tu presupuesto.
- Cómo desplegar sin pisar lo publicado.
- Un calendario de mantenimiento y una plantilla de auditoría.
- La ficha de una página que toda herramienta debería tener.

Parte 1 • Cuánto cuesta de verdad

En la guía 01 hicimos la cuenta de los tokens: unos US$1,40 al mes por 300 reseñas. Esa cifra es real, pero es **una parte** del costo. Una herramienta tiene economía propia, y hay que conocerla entera.

El costo completo

Concepto	En la tienda (ejemplo)	Cómo se mide
IA (tokens)	US$1,40 al mes	Registro de costos por corrida
Revisión humana	40 minutos al mes	Casos a revisión × tiempo por caso
Mantenimiento	~1 hora al mes	Ajustes, auditorías, cambios de lista
Publicación y servidor	US$0 (archivo de solo lectura)	Factura del servicio, si hay
Herramientas	Lo que ya pagas de tu asistente de IA	Planes vigentes

Contra 10 horas de trabajo manual, la herramienta sigue ganando por mucho. Pero fíjate en que el costo grande ya no son los tokens: **es el tiempo de personas**. Si un día la herramienta manda 100 casos a revisión en vez de 17, el costo real se multiplica aunque la factura de IA no cambie.

Registra cada corrida

No se puede gobernar lo que no se mide. Cada corrida anota cuánto procesó y cuánto costó:

```
import csv, datetime as dt, pathlib

def registrar_costo(corrida, unidades, usd):
    archivo = pathlib.Path("costos.csv")
    nuevo = not archivo.exists()
    with archivo.open("a", newline="") as f:
        w = csv.writer(f)
        if nuevo:
            w.writerow(["fecha", "corrida", "unidades", "usd", "usd_por_unidad"])
        w.writerow([dt.date.today().isoformat(), corrida, unidades,
                    round(usd, 4), round(usd / max(unidades, 1), 5)])

registrar_costo("informe-octubre", unidades=300, usd=1.32)
```

Con tres meses de registro ya sabes tu **costo por unidad** (por reseña, por informe, por caso), y eso es lo que te deja poner topes con sentido y, si algún día la ofreces a otros, ponerle precio.

Qué mueve el costo

Esto sorprende a casi todo el mundo. Vuelve a la cuenta de la guía 01:

Parte de cada llamada	Tokens	Peso
Prompt fijo (instrucciones + lista)	~700	85 %
La reseña	~120	15 %

El 85 % de lo que pagas por entrada son tus propias instrucciones, repetidas 300 veces. La reseña, que es lo que de verdad cambia, es la parte pequeña. De ahí salen las palancas para bajar el costo:

Palanca	Qué hacer	Ojo con
Prompt más corto	Quitar lo que no cambia el resultado	Medir con el set dorado antes y después (guía 02)
Parte repetida más barata	Algunos proveedores cobran menos por el inicio del prompt que se repite igual en cada llamada	Revisa si el tuyo lo ofrece y cómo se activa
Lotes no urgentes	Algunos proveedores tienen tarifas más bajas para trabajos que pueden esperar unas horas	El informe mensual no es urgente; una alerta sí
Modelo según la tarea	Uno más pequeño para clasificar, uno más capaz para redactar	Cambiar de modelo es un cambio: pasa por la compuerta (guía 03)
No volver a pagar	Guardar cada respuesta y reutilizarla (guía 02)	Que la huella incluya el modelo y el prompt

Los nombres, las tarifas y las condiciones de estas opciones cambian seguido: revísalas con tu proveedor el día que hagas la cuenta.

Lo pagado se reutiliza

La regla de la guía 02 aplica aquí con más fuerza: **cada corrida pagada se archiva**. Si cambias cómo se redacta el informe, vuelves a redactar sobre lo que ya tienes: no vuelves a recoger ni a clasificar. Solo se paga lo que cambió.

Si la vas a ofrecer a otros

Si la herramienta sirve para otras tiendas, el registro de costos es la base de tu precio. Una estructura sencilla:

Plan	Incluye	Tope	Si se pasa
Básico	Hasta 500 reseñas al mes, informe mensual	500	Se detiene y avisa, o se cobra un paquete adicional
Medio	Hasta 2.000 reseñas, informe mensual y alertas semanales	2.000	Igual

Cada plan lleva su **tope escrito**, igual que la herramienta lleva el suyo en el código. Un plan «ilimitado» con un costo por unidad que no es cero es una apuesta, no un precio.

Parte 2 • Claves que no se filtran

Para que tu herramienta hable con un servicio de IA necesitas una **clave de API**: un código largo que dice «esta llamada es de esta cuenta, cóbrale a ella». Es como la llave de tu casa y tu tarjeta de crédito en una sola cosa: quien la tiene, gasta a tu nombre.

Dónde viven las claves

- **En un archivo aparte**, normalmente llamado `.env`, fuera del código. Una clave por línea: `CLAVE_IA=...`.
- **Ese archivo nunca se comparte ni se sube.** Si usas git, va en `.gitignore`, la lista de lo que nunca se sube:

```
.env
cache/
datos/
corriendo.lock
```

- **En la nube, como secretos.** GitHub, los servicios de publicación y las plataformas de automatización tienen un lugar para guardar claves que no se ve en el código (lo usamos en la guía 03).
- **Nunca** en el código, en un chat, en un documento compartido, en una captura de pantalla ni en un correo.

Léelas sin ejecutarlas

Esto parece un detalle técnico y no lo es. Hay formas de cargar el archivo `.env` que lo **ejecutan** como si fuera un programa, y si una línea tiene un carácter raro, el error que sale en pantalla puede incluir la clave completa.

Me pasó trabajando con un asistente de IA: un comando que cargaba el archivo de claves falló y el mensaje de error mostró una clave entera, que quedó escrita en la conversación. Hubo que darla por perdida y cambiarla.

Desde entonces, el archivo de claves se **lee como texto** y se revisan solo las claves que espero. En pantalla solo aparecen los nombres y cuántos caracteres tiene cada una, nunca el valor:

```
import re, pathlib

ESPERADAS = {"CLAVE_IA", "CLAVE_CORREO"}

def leer_claves(ruta=".env"):
    claves = {}
    for linea in pathlib.Path(ruta).read_text().splitlines():
        m = re.match(r"^([A-Z_][A-Z0-9_]*)=(.*)$", linea.strip())
        if m and m.group(1) in ESPERADAS:
            claves[m.group(1)] = m.group(2).strip().strip('"').strip("'")
    for nombre in sorted(ESPERADAS):
        estado = f"{len(claves[nombre])} caracteres" if nombre in claves else "FALTA"
        print(f"{nombre}: {estado}")   # nunca el valor
    return claves
```

> **Si trabajas con un asistente de IA que ejecuta comandos:** dile desde el principio que nunca muestre, copie ni cargue tu archivo de claves con comandos que lo ejecuten, y que las claves las pegas tú en el archivo. Las herramientas de IA para programar son muy útiles, pero todo lo que sale en pantalla queda en la conversación.

Si una clave se filtra

No hay «a lo mejor nadie la vio». Si salió, se da por perdida:

PLANTILLA · PROCEDIMIENTO ANTE UNA CLAVE FILTRADA

1. **Detén** todo lo que use esa clave.
1. **Crea una clave nueva** en el panel del proveedor y **borra la vieja**. Borrarla es lo que de verdad cierra la puerta.
1. **Pon la nueva** en el `.env` y en los secretos de la nube. La pegas tú.
1. **Revisa el consumo** del proveedor desde el momento de la filtración: ¿hay llamadas que no reconoces?
1. **Busca dónde más quedó:** conversaciones, registros, capturas, el historial de git. Si quedó en git, borrarla del archivo no basta: sigue en el historial.
1. **Anota** qué pasó y qué cambias para que no se repita.

La segunda línea de defensa

Además de cuidar la clave, pon **límites de gasto y alertas** en el panel de tu proveedor de IA (la mayoría los tienen). Si todo lo demás falla, el límite del proveedor es lo que impide que una clave filtrada se convierta en una factura de cuatro cifras.

Parte 3 • Publicar sin abrir grifos

Llega el momento de que otros vean o usen la herramienta. Aquí está el error más caro de esta guía.

Lo que corre en el navegador de otro es público

Si haces una página web que llama directamente a la IA desde el navegador, la clave **tiene que estar en la página**. Y todo lo que está en una página web lo puede ver cualquiera que sepa abrir las herramientas del navegador. No importa si la escondes, si la partes en pedazos o si quitas el botón: ocultar algo en la pantalla no lo restringe.

```
MAL:   navegador de la persona (con tu clave adentro) ----------------> IA

BIEN:  navegador de la persona --> tu servidor ----------------------> IA
                                   (tiene la clave, valida, aplica topes)
```

La regla: **las restricciones viven en el servidor**. El servidor es la única parte que controlas tú. Ahí va la clave, ahí se valida lo que llega, ahí se aplican los topes y ahí se decide quién puede ver qué.

Un formulario abierto es un grifo

Supón que publicas una página donde cualquiera pega una reseña y la herramienta la clasifica. Funciona bonito. Hasta que alguien (o un programa automático) manda 50.000 reseñas en una noche. Cada una es una llamada a la IA, y cada llamada la pagas tú.

Una URL pública con un formulario abierto es **un grifo abierto a tu presupuesto**. Las defensas, de la más fuerte a la más débil:

Defensa	Qué es	Cuándo
No publicar el formulario	Publicar el **resultado** como archivo de solo lectura (un PDF, una página fija)	Casi siempre: la tienda no necesita que nadie clasifique nada, necesita leer el informe
Acceso controlado	Solo entran personas con usuario o con un enlace privado	Cuando de verdad otros tienen que usarla
Topes por persona y globales	Máximo de usos por persona al día y un máximo total	Siempre que haya acceso, además de lo anterior
Registro	Quién usó qué y cuándo	Siempre
Protección contra programas automáticos	Verificaciones que distinguen personas de programas	Si el formulario tiene que ser público

> **La pregunta que lo resuelve casi todo:** ¿la persona necesita **usar** la herramienta, o solo **ver lo que produce**? En la mayoría de los casos es lo segundo, y entonces no hay nada que proteger: publicas un archivo y listo.

Cada uno ve lo suyo

Si varias tiendas usan la herramienta, cada una debe ver **solo sus datos**. Esto tampoco se resuelve escondiendo cosas en la pantalla: se resuelve en la base de datos. Las bases de datos modernas permiten reglas por fila («esta tienda solo puede leer las filas que tienen su identificador») que se cumplen aunque alguien manipule la página. Y la clave con permiso para todo, la de servicio, vive solo en el servidor.

Antes de publicar algo con IA

PLANTILLA · LISTA PREVIA A PUBLICAR

- ¿Hay alguna clave en algo que corre en el navegador? Debe ser **no**.
- ¿Puede alguien que no conozco hacer que la IA trabaje (y yo pague)? Si es sí: ¿hay acceso controlado y topes?
- ¿Hay un tope global que detenga todo si se dispara el uso?
- ¿Puse límites de gasto y alertas en el panel del proveedor?
- ¿Cada usuario ve solo lo suyo, por reglas del servidor y no por la pantalla?
- ¿Queda registro de quién usó qué?
- ¿Podría publicar solo el resultado, en vez de la herramienta?

Parte 4 • Entregar sin pisar lo publicado

La entrega más simple que funcione

Antes de pensar en una aplicación web, pregúntate si alcanza con un archivo. Un informe en PDF, una hoja de cálculo o una página HTML que se abre sin internet resuelven muchísimo, y no hay que mantener servidores, claves ni usuarios.

Si entregas una página HTML, que sea **un solo archivo que funcione sin red**: las imágenes, las fuentes y los datos van adentro del archivo. Así se puede mandar, abrir en un avión y archivar, y dentro de dos años se sigue viendo igual. Un truco útil: que el programa que la genera **falle** si queda alguna referencia a algo externo, para que no se te escape.

Un detalle práctico: varios servicios de correo bloquean los adjuntos `.html`. Para mandarlo, compártelo por una carpeta en la nube o comprímelo.

Desplegar con verificación previa

Si la herramienta sí es una aplicación publicada, cada versión nueva pasa por una revisión **antes** de reemplazar la que está andando:

1. **La versión nueva se construye sin errores.** Si el paso de construir falla, no se publica nada.
2. **Pasan las pruebas y los filtros.** Las mismas de las guías 02 y 03.
3. **Se publica primero en una dirección de prueba.** La mayoría de servicios de publicación dan una URL temporal para cada versión antes de pasarla a la dirección real.
4. **Se revisa en esa dirección**, como la vería un usuario.
5. **Se pasa a la dirección real.**
6. **Sabes cómo volver atrás.** La mayoría de servicios guardan las versiones anteriores y permiten regresar a una en un clic. Averigua cómo antes de necesitarlo.

Error que ya vi: la copia que pisó el original

Me pasó: copié la carpeta de un proyecto publicado para hacer una variante, la cambié y la publiqué. Lo que no vi es que la carpeta copiada traía escondida la configuración que dice **a qué sitio se publica**. La variante se publicó encima del original y pisó una demostración que ya estaba andando.

La defensa es sencilla: **una carpeta, un sitio**. Si copias una carpeta, lo primero es revisar y cambiar a dónde publica. Y antes de cada publicación, una pregunta explícita:

```
echo "Vas a publicar en: $(cat .destino)"
read -p "¿Es el sitio correcto? (si/no) " r
[ "$r" = "si" ] || exit 1
```

`.destino` es un archivo de una línea donde escribes el nombre del sitio de esa carpeta. Tres líneas que te obligan a mirar antes de saltar.

Parte 5 • Mantener: lo que se pierde y lo que envejece

Una herramienta que funciona hoy no funciona sola para siempre. Hay dos enemigos: lo que **se pierde** y lo que **envejece**.

Lo que se pierde

- **Las carpetas temporales se borran.** Muchas herramientas (incluidos los asistentes de IA que programan) trabajan en carpetas temporales que se limpian solas. Me pasó: perdí varios programas útiles porque vivían en una de esas carpetas. Regla: lo que sirve más de una vez se copia a una carpeta permanente **con lo que necesita para correr** (la lista de librerías y versiones).
- **El código no vive en carpetas sincronizadas.** iCloud, Dropbox o OneDrive son buenísimos para documentos y malos para código: crean duplicados, bloquean archivos mientras sincronizan y algunas herramientas no los pueden leer. El código va en un repositorio con git, con una copia en un servicio como GitHub (privado si es tuyo).
- **Una sola fuente, copias que se sincronizan.** Si un prompt o una instrucción vive en varios lugares, se edita en **uno** y se copia a los demás. Si editas la copia, la próxima sincronización borra tu cambio.

Lo que envejece

Qué envejece	Ejemplo	Cada cuánto revisar
Precios	La tarifa por token cambió	Cada trimestre y al cambiar de modelo
Modelos	El proveedor retira la versión que usas	Cuando el proveedor avise; ten el nombre del modelo en la configuración, no regado por el código
Cifras que citas	Un dato del sector que ya se actualizó	Cada trimestre
Nombres de productos y servicios	Una herramienta cambió de nombre o dejó de existir	Cada trimestre
Reglas de las plataformas	Lo que permite o prohíbe una red social o un buscador	Cada trimestre
Tus categorías	El negocio cambió (guía 02)	Cada mes, con `no_mapeado`

Lo peligroso es que la herramienta no avisa que envejeció. Sigue respondiendo con la misma seguridad, solo que con datos de hace seis meses.

Auditar lo que ya está andando

Las instrucciones de una herramienta, sus prompts y sus documentos de apoyo **caducan** como cualquier entregable, y se auditan como cualquier entregable. Así lo hago:

1. **Parte el material en grupos** (por herramienta o por tema).
2. **Un agente por grupo**, en paralelo, busca cada afirmación verificable (cifras, nombres de productos, atribuciones, reglas de plataformas) y la contrasta con la **fuente primaria**: el sitio oficial, el documento original, el comunicado.
3. **Cada agente devuelve filas**, no un ensayo, con un reemplazo sugerido cuando algo caducó.
4. **Los agentes envían resultados parciales** a medida que avanzan, para no perder el trabajo si alguno se cuelga.
5. **Una persona decide y aplica.** El agente sugiere; quien es dueño de la herramienta decide.

La primera vez que lo hice sobre mis propias instrucciones de trabajo, corregí 19. Ninguna estaba «mal» cuando la escribí: el mundo había cambiado.

PLANTILLA · FILA DE AUDITORÍA

Archivo	Afirmación	Fuente primaria consultada	Fecha de consulta	Veredicto	Reemplazo sugerido
				vigente / caducó / no verificable	

«No verificable» también es un veredicto útil: si no encuentras la fuente de una cifra que estás citando, lo honesto es dejar de citarla.

El calendario de mantenimiento

Cada	Qué revisar	Tiempo
Semana	Bloqueos de filtros, corridas fallidas, gasto contra el tope	10 minutos
Mes	`no_mapeado`, `tema_sugerido`, costo por unidad, casos a revisión	30 minutos
Trimestre	Set dorado con casos nuevos, auditoría de afirmaciones, precios y modelos	2 a 3 horas
Cambio de modelo o de proveedor	Prueba mínima, set dorado completo, compuerta en cadena	Lo que tome; no se salta

La ficha de la herramienta

Si solo te llevas una cosa de esta guía, que sea esta: **toda herramienta tiene una ficha de una página**. Es lo que le entregas a quien la va a usar o a heredar, y lo primero que lees tú cuando la retomas en seis meses y no te acuerdas de nada.

PLANTILLA · FICHA DE LA HERRAMIENTA

Qué hace: en dos líneas, para alguien que no la conoce.

Dueño: quién decide y a quién se le pregunta.

Dónde vive: repositorio, carpeta, dónde se publica (y en qué dirección).

Cómo se corre: el comando o el botón, y cuándo corre sola.

Entradas y salidas: qué archivo recibe y qué entrega.

Costo: costo por unidad, tope por corrida, tope por mes, dónde está el registro.

Claves: nombres (nunca valores), dónde se guardan, quién las rota.

Semáforo: qué es verde, amarillo y rojo.

Calidad: última medición del set dorado y su fecha; umbral acordado.

Cómo se publica y cómo se vuelve atrás.

Mantenimiento: el calendario, y la fecha de la última auditoría.

Errores conocidos: lo que ya falló y cómo se resolvió.

Errores que ya vi

Error	Cómo se ve	Qué hacer
Contar solo los tokens	La herramienta «cuesta US$1» y se come horas de revisión	Costo completo, con tiempo de personas
No registrar el costo	No sabes tu costo por unidad ni dónde poner el tope	Registro por corrida
Prompt fijo largo sin medir	El 85 % de lo que pagas son tus instrucciones	Acortar y medir con el set dorado
Clave en el código o en el chat	Alguien gasta a tu nombre	`.env` fuera del código, en `.gitignore`, y secretos en la nube
Cargar el `.env` ejecutándolo	Un error imprime la clave	Leerlo como texto; en pantalla solo nombres y longitudes
Clave filtrada «que nadie vio»	Consumo que no reconoces	Rotar y borrar la vieja, siempre
Clave en la página web	Cualquiera la copia	Las restricciones viven en el servidor
Formulario abierto	Factura disparada	Publicar el resultado, no la herramienta; acceso y topes
Copiar una carpeta y publicar	Pisas lo que estaba publicado	Una carpeta, un sitio, y la pregunta antes de publicar
Código en carpetas temporales o sincronizadas	Programas perdidos o duplicados	Carpeta permanente y repositorio con git
Editar la copia	El cambio se borra en la siguiente sincronización	Editar en la fuente y sincronizar
No auditar	La herramienta responde con datos viejos y mucha seguridad	Auditoría trimestral contra fuentes primarias

Lista de verificación

- ☐ Conozco el costo completo: IA, revisión, mantenimiento, publicación y herramientas.
- ☐ Cada corrida registra unidades y costo, y sé mi costo por unidad.
- ☐ Sé qué parte del costo es el prompt fijo y lo medí al acortarlo.
- ☐ Revisé con mi proveedor las opciones para abaratar la parte repetida y los trabajos no urgentes.
- ☐ Lo pagado se archiva y se reutiliza.
- ☐ Si la ofrezco a otros, cada plan tiene su tope escrito.

- ☐ Las claves viven en `.env`, fuera del código, y `.env` está en `.gitignore`.
- ☐ En la nube, las claves son secretos.
- ☐ El `.env` se lee como texto; en pantalla solo salen nombres y longitudes.
- ☐ Tengo escrito el procedimiento ante una clave filtrada.
- ☐ Hay límites de gasto y alertas en el panel del proveedor.
- ☐ Ninguna clave corre en el navegador; las restricciones están en el servidor.

- ☐ No hay formularios abiertos; si hay acceso, hay topes por persona y globales.
- ☐ Cada usuario ve solo lo suyo, por reglas del servidor.
- ☐ Me pregunté si alcanza con publicar el resultado en un archivo.
- ☐ Los entregables HTML son un solo archivo que funciona sin red.
- ☐ Cada versión pasa por construcción, pruebas, dirección de prueba y revisión antes de la real.
- ☐ Sé cómo volver a la versión anterior.

- ☐ Una carpeta, un sitio; y pregunto el destino antes de publicar.
- ☐ Lo reutilizable vive en una carpeta permanente con lo que necesita para correr.
- ☐ El código está en un repositorio con git, no en una carpeta sincronizada.
- ☐ El nombre del modelo está en la configuración.
- ☐ Tengo calendario de mantenimiento y la fecha de la última auditoría.
- ☐ La herramienta tiene su ficha de una página.

Las cuatro guías en una tabla

Con esta guía se cierra la serie. Este es el recorrido completo:

Guía	La pregunta	La idea central
01 · Diseña tu primera herramienta	¿Cómo paso de «lo hago a mano» a una herramienta diseñada?	La IA propone; las reglas deciden. Se diseña antes de construir
02 · Que la IA no invente	¿Cómo demuestro que funciona?	Listas cerradas con definiciones, salidas verificables, set dorado y filtros que bloquean
03 · Que corra solo	¿Cómo la automatizo sin perder el control?	Pasos con archivos, semáforo, topes en código y cambios por compuerta
04 · Entregar y mantener	¿Cómo la entrego y la mantengo viva?	Costo completo, claves seguras, publicar el resultado y auditar lo que envejece

De dónde sale esto

Todo lo de esta guía lo he aplicado en herramientas reales de estrategia y marketing, publicadas y en producción. El ejemplo de la tienda de café y sus números son **ficticios**. Los tres errores que cuento (la clave que salió en un error, la copia que pisó una demostración publicada y los programas perdidos en carpetas temporales) me pasaron de verdad, y los cuento para que no te pasen a ti.

Los nombres de servicios, sus planes y sus tarifas cambian seguido. Las prácticas de esta guía no dependen de ninguno en particular: revisa lo vigente con tu proveedor antes de decidir.

No soy programador; vengo de la estrategia. Empecé a armar herramientas con IA sin saber por dónde empezar, y estas cuatro guías son lo que me habría gustado tener ese primer día. Si alguien de ingeniería haría algo distinto, me sirve que me lo diga.

Si la serie te sirvió, guárdala y pásasela a alguien que esté a punto de construir su primera herramienta. Y si construyes algo con esto, me encantaría saber qué fue.